

Agent Evaluation Lab

Deployment Manual

May 2026

1. Audience Definition

This deployment documentation is intended for:

- system administrators responsible for preparing and running the platform
- IT personnel validating local or hosted deployments
- developers setting up the repository for testing, demo, or extension
- instructors, reviewers, or evaluators who need to reproduce the project workflow

The document is written so that a new technical user can deploy the project without prior knowledge of the codebase.

2. System Overview

Agent Evaluation Lab includes three primary runtime components:

1. Python evaluation framework

Provides the CLI, scenario execution engine, grading logic, reporting, release-gate checks, and evaluator service.

2. Built-in RAG target agent

Exposes a local HTTP interface used for demo and end-to-end evaluation.

3. Next.js platform

Provides the sandbox, dashboard, evaluation workflow, release gate, evidence downloads, regression views, and monitoring features.

Local Demo Ports

- 8000 - built-in RAG target agent
- 3000 - Next.js platform
- 5433 - PostgreSQL database through Docker

3. Prerequisite Installation

Install the following before deploying the software on any platform:

- Python 3.11 or newer
- Node.js 18 or newer
- npm
- Git
- Docker with Docker Compose support
- OpenAI API key

Optional but useful:

- uv for Python environment management
- PostgreSQL client tools

Windows Prerequisites

Install:

- Python 3.11+
- Node.js 18+
- Git for Windows
- Docker Desktop

macOS Prerequisites

Install:

- Python 3.11+
- Node.js 18+
- Git
- Docker Desktop

Linux Prerequisites

Install:

- Python 3.11+
- Node.js 18+
- Git
- Docker Engine
- Docker Compose plugin

Example packages for Ubuntu or Debian:

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3-pip nodejs
npm git docker.io docker-compose-plugin
```

4. Repository Setup

Clone the repository:

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
```

Optional verification:

```
git status --short --branch
```

5. Configuration Instructions

5.1 Root Python Environment

Create the root environment file:

```
cp .env.example .env
```

Set at least the following values:

```
OPENAI_API_KEY=sk-your-real-key
RAG_MODEL=gpt-4o-mini
RAG_TEMPERATURE=0.1
RAG_MAX_TOKENS=512
RAG_HOST=127.0.0.1
RAG_PORT=8000
```

You may also set the API key in the current shell session:

```
export OPENAI_API_KEY="sk-your-real-key"
```

5.2 Platform Environment

Create the platform environment file:

```
cd platform
cp .env.example .env
cd ..
```

For local deployment with the included PostgreSQL container, use:

```
DATABASE_URL="postgresql://agent_eval:agent_eval_password@localhost:5433/agent_eval?schema=public"
OPENAI_API_KEY=sk-your-real-key
PYTHON_CLI_PATH=../.venv/bin/python
AGENT_EVAL_PATH=../agent_eval_lab
RBAC_DEFAULT_ROLE=admin
RBAC_STRICT=false
PUBLIC_SELF_SERVE_MODE=false
```

Configuration notes:

- `DATABASE_URL` points the Next.js platform to the Docker PostgreSQL service.
- `OPENAI_API_KEY` is required for the built-in RAG service and evaluation flows.
- placeholder Clerk keys may remain unchanged for local development.
- the application uses development authentication fallback outside production.
- do not commit `.env` or `platform/.env`.

6. Platform-Specific Deployment Instructions

6.1 Windows Deployment

Clone the repository:

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
```

Create and activate a virtual environment:

```
py -3.11 -m venv .venv
.venv\Scripts\activate
```

Install Python dependencies:

```
python -m pip install -e "[dev]"
```

Create the root environment file:

```
copy .env.example .env
```

Set the OpenAI key for the session:

```
$env:OPENAI_API_KEY="sk-your-real-key"
```

Configure the platform environment and start PostgreSQL:

```
cd platform
copy .env.example .env
docker compose up -d postgres
npm install
npm run db:generate
npm run db:push
```

Start the platform:

```
npm run dev
```

In another terminal, start the RAG service:

```
cd agent-eval-lab
.venv\Scripts\activate
python -m agent_eval_lab.rag_service.server
```

Open:

- <http://127.0.0.1:8000>
- <http://127.0.0.1:3000/sandbox>

6.2 macOS Deployment

Clone the repository:

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
```

Create and activate a virtual environment:

```
python3.11 -m venv .venv
source .venv/bin/activate
```

Install Python dependencies:

```
python -m pip install -e ".[dev]"
```

Create the root environment file and set the API key:

```
cp .env.example .env
export OPENAI_API_KEY="sk-your-real-key"
```

Configure and start the platform:

```
cd platform
cp .env.example .env
docker compose up -d postgres
npm install
npm run db:generate
npm run db:push
npm run dev
```

In another terminal, start the RAG service:

```
cd ..
source .venv/bin/activate
python -m agent_eval_lab.rag_service.server
```

Open:

- <http://127.0.0.1:8000>
- <http://127.0.0.1:3000/sandbox>

6.3 Linux Deployment

Install prerequisites if needed:

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3-pip nodejs
npm git docker.io docker-compose-plugin
```

Clone the repository:

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
```

Create and activate a virtual environment:

```
python3.11 -m venv .venv
source .venv/bin/activate
```

Install Python dependencies:


```
python -m pip install -e ".[dev]"
```

Create the root environment file and set the API key:

```
cp .env.example .env  
export OPENAI_API_KEY="sk-your-real-key"
```

Configure and start the platform:

```
cd platform  
cp .env.example .env  
docker compose up -d postgres  
npm install  
npm run db:generate  
npm run db:push  
npm run dev
```

In another terminal, start the RAG service:

```
cd ..  
source .venv/bin/activate  
python -m agent_eval_lab.rag_service.server
```

Open:

- `http://127.0.0.1:8000`
- `http://127.0.0.1:3000/sandbox`

6.4 Common Platform Notes

- Docker must be running before `docker compose up -d postgres`
- the platform expects PostgreSQL on `localhost:5433`

- the built-in RAG agent runs on `127.0.0.1:8000`
- the platform runs on `127.0.0.1:3000`
- the OpenAI API key must be available before starting the RAG service or running CLI evaluations

7. Deployment Scripts or Code Snippets

The following commands are the main deployment commands used for local setup and verification.

7.1 Python Setup

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
```

7.2 Platform Setup

```
cd platform
cp .env.example .env
docker compose up -d postgres
npm install
npm run db:generate
npm run db:push
npm run dev
```

7.3 Start the Built-In RAG Service

```
cd ..
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
python -m agent_eval_lab.rag_service.server
```

7.4 Run CLI Evaluation

```
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
agent-eval run-all-scenarios --config-file
```

```
examples/rag_agent_config.yaml
```

7.5 Run Release Gate Check

```
source .venv/bin/activate
agent-eval check-gate \
  --report-file reports/evaluation_report.json \
  --policy-file policy/release_gate.enterprise.yaml
```

8. Running the Software Locally

Open three terminals.

Terminal 1: RAG Target Agent

```
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
python -m agent_eval_lab.rag_service.server
```

Health check:

```
curl -i http://127.0.0.1:8000/health
```

Terminal 2: Next.js Platform

```
cd platform
npm run dev
```

Terminal 3: Optional CLI Verification

```
source .venv/bin/activate
agent-eval run-all-scenarios --config-file
```

```
examples/rag_agent_config.yaml
```

Main URLs:

- <http://127.0.0.1:8000/>
- <http://127.0.0.1:3000/sandbox>
- <http://127.0.0.1:3000/dashboard>
- <http://127.0.0.1:3000/dashboard/evaluations/new>

9. Testing and Troubleshooting

9.1 Testing the Deployment

Test the target agent directly:

```
curl -i http://127.0.0.1:8000/health
```

Test the sandbox flow:

1. open </sandbox>
2. set base URL to <http://127.0.0.1:8000>
3. run auto-detect
4. run health check
5. send a test query

Test the evaluation workflow:

1. open </dashboard/evaluations/new>
2. select HTTP agent
3. use base URL <http://127.0.0.1:8000>
4. select scenarios
5. start the evaluation

6. verify status and results on the detail page

Test the reports:

- download JSON report
- download Markdown report
- download evidence artifact

9.2 Validation Commands

Python tests:

```
source .venv/bin/activate
pytest
```

Platform build:

```
cd platform
npm run build
cd ..
```

CLI smoke run:

```
source .venv/bin/activate
agent-eval run-all-scenarios --config-file
examples/rag_agent_config.yaml
```

9.3 Common Troubleshooting Issues

Issue: Python version is too old

Fix:

```
python3.11 --version
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"
```

Issue: `pip` command is not found

Fix:

```
python3.11 -m pip install -e ".[dev]"
```

Issue: Port `8000` is already in use

Fix:

```
lsof -i :8000
kill <PID>
```

Issue: Prisma cannot reach the database on `localhost:5433`

Fix:

```
cd platform
docker compose up -d postgres
npm run db:push
docker compose ps
cd ..
```

Issue: Sandbox health check fails

Verify:

- the RAG service is running
- `OPENAI_API_KEY` is set
- the base URL is `http://127.0.0.1:8000`

Retest:

```
curl -i http://127.0.0.1:8000/health
```

Issue: Platform styling or static files look broken

Fix:

```
cd platform
npm run dev:clean
cd ..
```

Then hard refresh the browser.

Issue: Evaluation remains queued or running

Check:

- target agent availability
- database availability
- API key configuration
- platform refresh behavior

Retry the run from the UI if the failure was transient.

Issue: Report download fails

Test the endpoint directly:

```
curl -i  
"http://127.0.0.1:3000/api/evaluations/{id}/download?format=json"
```

Replace `{id}` with the actual evaluation ID.

10. Production Deployment Model

The recommended production architecture separates the web platform from the long-running evaluator service.

10.1 Platform Deployment

Deploy the `platform/` directory to Vercel.

Required environment variables:

```
DATABASE_URL=postgresql://...  
OPENAI_API_KEY=sk-...  
EVAL_RUNNER_URL=https://your-runner-service.example.com/v1/evaluate  
EVAL_RUNNER_TOKEN=replace-with-shared-secret  
EVAL_EXECUTION_MODE=inline  
API_KEY_HASH_SALT=replace-with-random-hex
```

Optional public self-serve variables:

```
PUBLIC_SELF_SERVE_MODE=true  
PUBLIC_SELF_SERVE_DAILY_LIMIT=3  
PUBLIC_SELF_SERVE_MAX_SCENARIOS=3  
PUBLIC_SELF_SERVE_SALT=replace-with-random-hex
```


Optional audit and evidence variables:

```
SLACK_WEBHOOK_URL=  
JIRA_WEBHOOK_URL=  
AUDIT_SIGNING_KEY=replace-with-long-random-secret  
AUDIT_SIGNING_KEY_ID=audit-2026-q1  
EVIDENCE_SIGNING_KEY=replace-with-long-random-secret  
EVIDENCE_SIGNING_KEY_ID=key-2026-q1
```

Apply the production schema:

```
cd platform  
DATABASE_URL="postgresql://..." npm run db:push  
cd ..
```

10.2 Evaluator Runner Deployment

Deploy the Python runner service on a long-running host.

Start command:

```
uvicorn agent_eval_lab.runner.service:app --host 0.0.0.0 --port  
8080
```

Required environment variables:

```
OPENAI_API_KEY=sk-...  
EVAL_RUNNER_TOKEN=replace-with-shared-secret  
SCENARIO_DEFINITIONS_DIR=scenario_definitions
```

Health check:

```
curl https://your-runner-service.example.com/health
```

10.3 Platform-to-Runner Connection

Set the following in the platform deployment:

```
EVAL_RUNNER_URL=https://your-runner-service.example.com/v1/evaluate  
EVAL_RUNNER_TOKEN=replace-with-shared-secret
```

The token must match the runner service configuration.

11. Maintainer Notes

Before submission or handoff:

- run validation commands
- capture the required screenshots
- confirm `.env`, `platform/.env`, generated reports, `.next`, and local exports are not staged
- confirm README and deployment instructions match the actual repository behavior